

5. Data Management and Resilience

The EcoGrid Energy platform needs to manage different types of data. These include frequent smart-meter readings, energy trading information, and financial settlement records. Each type of data is handled by a different bounded context, so the system needs to keep the data reliable without making all parts of the platform dependent on one database or service.

For this reason, EcoGrid does not use one large transaction across the whole platform. Each bounded context manages its own data and completes its own local transaction. When a business process needs more than one context, the system uses events and controlled coordination between them. Unique transaction identifiers, retries, and compensating actions are also used to deal with failures.

This approach supports the scalability and loose coupling requirements discussed earlier in the report. It also supports important quality attributes such as reliability, availability, performance, and maintainability [1].

5.1 Distributed Transactions

A good example of a distributed transaction in EcoGrid is the process of settling an energy trade. For example, a customer may purchase 10 kWh from another user. The Marketplace Context needs to record the trade, while the Financial Settlement Context needs to create the associated financial transaction.

Using one distributed database transaction for both contexts could make the services highly dependent on each other and could reduce scalability. Instead, the proposed design uses the **Saga pattern**. The Saga breaks the overall business process into smaller local transactions. Each context completes its own transaction before the next part of the process is started. If a later step fails, the system can retry the operation or perform a compensating action where necessary [3].

A simplified process is:

Marketplace → Trade Created → Settlement Requested → Financial Transaction Updated → Settlement Confirmed

When the Marketplace creates a trade, it assigns a unique Trade Id and stores the trade in its database. It then sends a settlement request event. The Financial Settlement context receives the event and processes the financial part of the transaction. After the settlement is successfully completed, it sends a SettlementCompleted event.

If the settlement fails, EcoGrid does not need to start the whole process again. A temporary failure can be retried, while a permanent failure can result in the trade being placed into a failed or pending state. This allows the system to maintain a consistent business state without requiring one large transaction across multiple services [3].

Transactional Outbox

There is a possible reliability problem when a database update and event publishing are performed as two separate operations. For example, the Marketplace could save a trade

successfully and then the application could crash before sending the SettlementRequested event.

The **Transactional Outbox pattern** can be used to reduce this problem. When the Marketplace saves the trade, it also saves the event information in an outbox table within the same local database transaction. A separate background process can then read the event from the outbox and publish it to the message broker.

Therefore, even if the application fails immediately after the database transaction, the event is still stored and can be published later. This pattern helps ensure that an important business event is not lost between the database update and message publication [7].

Idempotency

Another important issue is duplicate messages. In an event-driven system, a message may sometimes be delivered more than once because of retries or communication failures. This is especially important for financial transactions because processing the same settlement twice could result in an incorrect balance or duplicate ledger entry.

EcoGrid should therefore use **idempotent processing** for settlement operations. Each transaction should have a unique identifier such as a TradeID or TransactionID.

SettlementRequested(TransactionID = TX10025)

Before processing the request, the Financial Settlement service checks whether TX10025 has already been processed. If it has, the service can return the existing result rather than creating another financial entry.

This means that receiving the same message again will not cause the financial operation to be performed twice. Idempotency is therefore important when using retries and at-least-once message delivery [3], [8].

The distributed transaction approach can be summarised as follows:

Requirement	Design Approach
Marketplace transaction	Local ACID transaction
Financial transaction	Local ACID transaction
Cross-context coordination	Saga pattern
Reliable event publication	Transactional Outbox

Duplicate prevention	Idempotent processing
Temporary failure	Controlled retry
Permanent failure	Compensating action or failed state
Transaction identification	Unique TradeID/TransactionID

Overall, this approach allows the Marketplace and Financial Settlement contexts to remain separate while still providing a reliable way to complete transactions that involve both contexts.

5.2 Fault Tolerance

Fault tolerance is important in EcoGrid because the Smart Meter Integration context may continuously receive a large number of readings. If the smart-meter processing service stops working, this should not cause the Marketplace or Financial Settlement services to stop.

The event-driven architecture helps achieve this separation. Instead of depending entirely on direct service-to-service communication, messages can be stored in a message broker and processed independently. This means that one component can experience a problem without immediately affecting the rest of the platform [2].

Smart Meter Fault Tolerance

The Smart Meter Integration context should use a durable message broker so that incoming meter readings are not lost if a processing service becomes unavailable. The messages can remain in the broker until a worker is available to process them successfully.

More than one ingestion worker can also be deployed:

Smart Meter → Message Broker → Worker 1 / Worker 2 / Worker 3 → Time-Series Database

If Worker 1 fails, Workers 2 and 3 can continue processing the available messages. Additional workers can also be added when the number of incoming readings increases.

This provides both fault tolerance and scalability. It also prevents a single processing instance from becoming a single point of failure [2].

Retry and Dead-Letter Handling

Some failures in EcoGrid will only be temporary. For example, a database connection might briefly fail or there could be a temporary network problem. In these situations, the system should attempt the operation again instead of immediately treating the message as permanently failed.

However, continuously retrying a failed message could create additional load on the system. EcoGrid should therefore use:

- a limited number of retry attempts;
- exponential backoff between retries;
- message acknowledgement after successful processing;
- a dead-letter queue for messages that repeatedly fail; and
- monitoring and alerts for unusual failure rates.

A Dead-Letter Queue (DLQ) is useful for messages that cannot be processed automatically. A malformed meter reading could be moved to the DLQ rather than being retried indefinitely. Developers can then investigate the message and determine whether it needs to be corrected or discarded.

Circuit Breaker

Some communication within EcoGrid may still need to be synchronous. For example, the Marketplace may need to communicate with the Financial Settlement service for a particular operation.

A **Circuit Breaker** can help protect the system when the Settlement service is unavailable. Without a circuit breaker, the Marketplace could continue sending requests to a service that is already failing. This could waste resources and potentially increase the effect of the failure.

When repeated failures are detected, the circuit breaker temporarily stops requests to the unavailable service. After some time, it can allow requests again to check whether the service has recovered.

This approach helps prevent a failure in one service from spreading to other parts of the system. Retry and Circuit Breaker patterns are commonly used to deal with temporary failures in distributed systems [3].

Database Resilience

Database failures also need to be considered. Depending on the requirements of each bounded context, EcoGrid can use:

- database replication;
- automated backups;
- point-in-time recovery;
- transaction logs;
- database health checks; and
- controlled database connection pools.

The Financial Settlement database requires additional attention because financial records need to remain accurate and auditable. Instead of simply overwriting previous financial information, the system should maintain a history of transactions so that previous activities can be checked if a problem occurs.

The overall fault-tolerance approach is intended to make sure that a failure in one part of EcoGrid does not automatically make the whole platform unavailable.

5.3 Observability

EcoGrid contains several bounded contexts and uses asynchronous communication between some of them. Because of this, it may be difficult to understand what happened to a particular transaction by looking at one application's logs alone.

Observability should therefore be considered as part of the system design. The three main areas are:

1. **Logs**
2. **Metrics**
3. **Distributed tracing**

These provide different types of information about the health and behaviour of the platform. Together, they can help developers find errors and performance problems more quickly [9].

Centralised Logging

Each EcoGrid component should produce structured logs. Some useful information to include is:

- timestamp;
- service or context name;
- TradeID;
- TransactionID;
- EventID;
- correlation ID;
- operation;
- success or failure status; and
- error details.

CorrelationID: COR-78125
TradeID: TR-10045
Service: FinancialSettlement
Event: SettlementRequested
Status: Failed
Reason: PaymentServiceTimeout

Having these logs in a central location makes it easier to investigate an operation that has passed through several different services.

Correlation IDs

A **Correlation ID** created when a business operation starts and then included in the related events and service requests.

Marketplace
COR-78125 → TradeCreated
↓
Message Broker
COR-78125 → SettlementRequested
↓
Financial Settlement
COR-78125 → SettlementCompleted

Using the same ID throughout the process allows developers to follow a single operation across different services. This is especially useful in an event-driven system because the whole process does not occur within one synchronous request [2].

Metrics

EcoGrid should monitor both system performance and business-related activity.

Smart Meter Integration

- readings received per second;
- processing latency;
- message backlog;
- failed readings;
- retry count; and
- dead-letter queue size.

Marketplace

- number of active trades;
- trade-processing latency;
- failed trade requests; and
- matching-engine response time.

Financial Settlement

- successful settlements;
- failed settlements;
- duplicate transaction attempts;
- settlement processing time; and
- pending transactions.

These metrics can help the team identify unusual behaviour before it becomes a larger issue.

Distributed Tracing

Distributed tracing is useful for operations that move through multiple components. For example, an energy trade may follow this path:

API → Marketplace → Message Broker → Financial Settlement → Payment Provider

If a trade normally takes around 500 milliseconds but one transaction takes three seconds, tracing can help identify which part of the process caused the delay.

This can be particularly helpful for the EcoGrid development team because it reduces the need to manually check multiple services when investigating an issue.

Health Checks and Alerts

The major EcoGrid components also provide health information. Monitoring tools can use this information to generate alerts when something starts behaving abnormally.

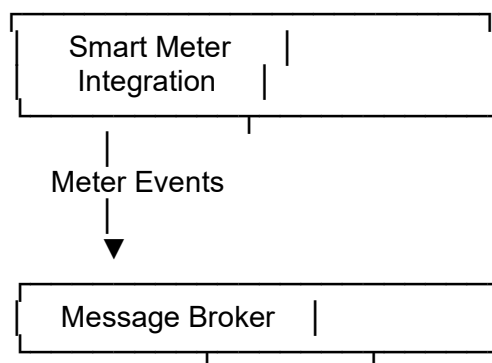
Metric	Example Alert Condition
Smart-meter backlog	Continuously increasing
Event-processing latency	Above defined threshold
Dead-letter messages	Sudden increase
Settlement failures	Higher than normal
Database connections	Repeated failures
API response time	Significant increase
Service availability	Health check failure

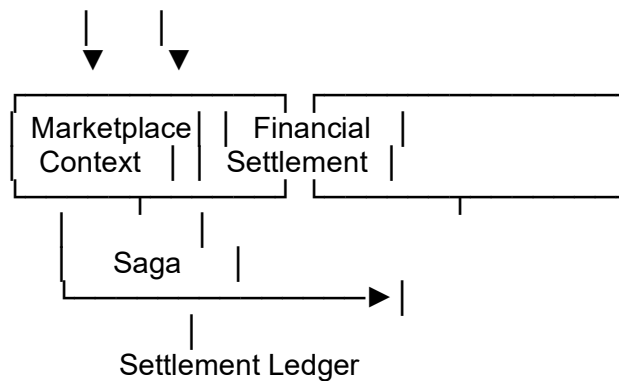
These alerts allow the development team to investigate problems before they have a major impact on users.

Therefore, observability is not only useful for troubleshooting. It also supports the availability, performance, reliability, and maintainability requirements of the EcoGrid architecture [1].

5.4 Overall Data Management and Resilience Strategy

The proposed design connects the three main bounded contexts while keeping their responsibilities separate.





The main design decisions are:

- Local transactions are used inside each bounded context.
- The Saga pattern coordinates processes that involve more than one context.
- The Transactional Outbox reduces the possibility of losing important events.
- Idempotency prevents duplicate processing when messages are retried.
- The message broker separates high-volume smart-meter processing from other workloads.
- Controlled retries are used for temporary failures.
- Dead-letter queues handle messages that repeatedly fail.
- Circuit breakers protect services from continuously calling unavailable dependencies.
- Backups and replication improve database resilience.
- Centralised logging provides information for troubleshooting.
- Metrics allow both technical and business performance to be monitored.
- Distributed tracing helps track operations across different services.
- Correlation IDs make it easier to connect related events and logs.

This design supports the original EcoGrid architecture by allowing the Smart Meter Integration, Marketplace, and Financial Settlement contexts to operate and scale independently. At the same time, it provides mechanisms for dealing with transaction failures, duplicate messages, service outages, and database problems.

The design also recognises that not every part of the system needs strong consistency at all times. Event-driven communication may result in eventual consistency between some contexts, which is acceptable where immediate consistency is not essential. Stronger consistency should instead be prioritised for areas where incorrect data could have a serious business impact, particularly financial settlement [2].

Overall, the proposed approach provides a practical balance between scalability, reliability, data integrity, and maintainability for the EcoGrid Energy platform.